

WIPRO CAPSTONE PROJECT MODULE 2

Analysis of Real-Time Object Detection Using YOLO

for Autonomous Vehicle Perception

Submitted By

Parthib Ranjan Sarbajna

Enrollment Number: 12023002028150

Registration Number: 304202300901068

Institute of Engineering and Management, Kolkata

B.Tech in CSE(AI & ML)

Project Report

2026

Abstract

Autonomous vehicles require perception systems capable of identifying and localizing objects in their surrounding environment. Object detection is therefore an important component of autonomous vehicle perception because it enables a system to identify objects such as vehicles and pedestrians from visual data.

This project presents an implementation and analysis of **YOLO11n** for object detection in an urban vehicle scene. The system was implemented using Python and the Ultralytics YOLO framework and executed using Google Colab with an NVIDIA Tesla T4 GPU.

The implemented system accepts an image as input and performs object detection to produce bounding boxes, object classes and confidence scores. In the experimental vehicle scene, the model detected a bus with a confidence score of approximately 0.940 and multiple pedestrians.

The project also evaluates inference performance. A single measured inference produced an inference time of 0.0626 seconds, corresponding to approximately 15.97 FPS. A separate 20-run benchmark produced an average inference time of 0.0202 seconds, with a minimum of 0.0184 seconds and a maximum of 0.0249 seconds. The resulting average throughput was approximately 49.53 FPS in the tested environment.

The report further discusses YOLO, SSD and Faster R-CNN from the perspective of architecture and real-time object detection. The experimental evaluation in this project is limited to YOLO11n, while the comparison with SSD and Faster R-CNN is presented as a qualitative discussion.

Contents

1	Introduction	7
1.1	Background	7
1.2	Object Detection in Autonomous Vehicles	7
1.3	Project Motivation	8
1.4	Problem Statement	8
1.5	Project Objectives	8
2	Technical Background	10
2.1	YOLO	10
2.2	YOLO11n	10
2.3	Bounding Boxes	11
2.4	Confidence Score	11
2.5	Inference Time	11
3	YOLO, SSD and Faster R-CNN	13
3.1	Overview	13
3.2	YOLO	13
3.3	SSD	13
3.4	Faster R-CNN	14
3.5	Qualitative Comparison	14
3.6	Experimental Scope	14
4	Methodology	16
4.1	System Architecture	16

4.2	Input Image	16
4.3	Detection Process	16
4.4	Performance Measurement	17
5	Experimental Setup	18
5.1	Hardware Environment	18
5.2	Software Components	18
5.3	Project Structure	19
6	Results and Analysis	20
6.1	Object Detection Result	20
6.2	Detection Visualization	20
6.3	Detection Confidence Results	21
6.4	Confidence Score Visualization	22
6.5	Single Inference Performance	22
6.6	20-Run Performance Benchmark	23
6.7	Inference Time Visualization	23
6.8	FPS Performance	24
6.9	Object Distribution	24
7	Discussion	26
7.1	Detection Performance	26
7.2	Real-Time Considerations	26
7.3	YOLO and SSD	27
7.4	YOLO and Faster R-CNN	27
7.5	Suitability for Autonomous Vehicle Perception	27
8	Limitations	29
8.1	Limited Dataset	29
8.2	No Ground-Truth Accuracy Evaluation	29
8.3	Hardware Dependency	29

8.4	Image-Based Evaluation	30
8.5	Limited Model Comparison	30
9	Future Scope	31
9.1	Larger Autonomous Driving Datasets	31
9.2	Video-Based Detection	31
9.3	Object Tracking	31
9.4	Full Model Comparison	32
9.5	Edge Hardware Deployment	32
9.6	Sensor Fusion	32
10	Conclusion	33
A	Experimental Results Summary	35
B	Project Repository	36

List of Figures

6.1	YOLO11n detection result for the urban vehicle scene	21
6.2	Confidence scores of detected objects	22
6.3	Inference time measurements	23
6.4	FPS performance during repeated inference	24
6.5	Distribution of detected object classes	25

List of Tables

3.1	Qualitative comparison of object detection approaches	14
5.1	Experimental environment	18
6.1	Detected objects and confidence scores	22
6.2	20-run YOLO11n performance benchmark	23
A.1	Complete experimental summary	35

Chapter 1 Introduction

1.1 Background

Autonomous vehicles rely on perception systems to understand the environment surrounding the vehicle. These systems process data from different sensors, including cameras, LiDAR and radar.

Among these sensors, cameras provide rich visual information that can be processed using computer vision and deep learning techniques. Object detection is one of the important computer vision tasks used for interpreting camera data.

Object detection performs two major tasks:

1. Identification of the class of an object.
2. Localization of the object using a bounding box.

For autonomous vehicles, object detection can help identify road users such as cars, buses, pedestrians, motorcycles and bicycles.

1.2 Object Detection in Autonomous Vehicles

A simplified autonomous vehicle perception pipeline can be represented as:

Camera Input $\rightarrow$ Preprocessing $\rightarrow$ Object Detection $\rightarrow$ Object Classification $\rightarrow$ Decision Support
(1.1)

An autonomous vehicle operates in a continuously changing environment. Therefore, perception systems need to process visual information with sufficiently low latency.

Consequently, both detection quality and computational performance are important when selecting an object detection model.

1.3 Project Motivation

Traditional computer vision approaches often depend on manually engineered features and may have difficulty handling complex visual conditions.

Deep-learning-based object detection models can automatically learn visual features from training data.

YOLO, or You Only Look Once, is a family of object detection models that performs object localization and classification within a unified detection pipeline.

The ability to perform detection efficiently makes YOLO relevant to applications where low inference latency is important.

1.4 Problem Statement

The objective of this project is to implement and analyze a YOLO-based object detection system for an autonomous vehicle perception scenario.

The project focuses on:

- Object detection using YOLO.
- Detection confidence analysis.
- Bounding-box visualization.
- Inference-time measurement.
- Approximate FPS measurement.
- Discussion of real-time suitability.
- Comparison of YOLO with SSD and Faster R-CNN at a conceptual level.

1.5 Project Objectives

The major objectives of the project are:

1. Implement YOLO11n for object detection.
2. Process an urban vehicle scene.

3. Detect vehicles and pedestrians.
4. Generate annotated detection results.
5. Record object classes and confidence scores.
6. Measure inference latency.
7. Estimate inference FPS.
8. Analyze the real-time characteristics of YOLO11n.
9. Discuss YOLO, SSD and Faster R-CNN.

Chapter 2 Technical Background

2.1 YOLO

YOLO stands for *You Only Look Once*. It is a family of deep-learning-based object detection models.

YOLO treats object detection as a unified prediction problem. Instead of separately performing multiple traditional stages, the model processes an image through a neural network and predicts object locations and classes.

A simplified representation of the detection process is:

$$I \rightarrow F_{\theta}(I) \rightarrow \{B_i, C_i, S_i\} \quad (2.1)$$

where:

- I is the input image.
- F_{θ} represents the trained neural network.
- B_i represents the bounding box of object i .
- C_i represents the predicted object class.
- S_i represents the confidence score.

2.2 YOLO11n

This project uses the lightweight **YOLO11n** model.

The “n” configuration is a smaller YOLO model variant designed to require fewer computational resources than larger variants.

A lightweight model can be useful in scenarios where inference speed and computational efficiency are important considerations.

2.3 Bounding Boxes

Object detection models identify objects by predicting bounding boxes.

A bounding box can be represented by:

$$B = (x, y, w, h) \quad (2.2)$$

where:

- x and y represent the position of the bounding box.
- w represents its width.
- h represents its height.

2.4 Confidence Score

The confidence score represents the model's confidence in a detected object.

A detection can be represented as:

$$D_i = (x_i, y_i, w_i, h_i, c_i, s_i) \quad (2.3)$$

where c_i is the predicted class and s_i is the confidence score.

The implementation uses a confidence threshold during inference to filter low-confidence predictions.

2.5 Inference Time

Inference time represents the amount of time required by the model to process an input image.

If T is the inference time in seconds, approximate FPS can be calculated using:

$$FPS = \frac{1}{T} \quad (2.4)$$

For example, an average inference time of 0.0202 seconds produces:

$$FPS \approx \frac{1}{0.0202} \approx 49.53 \quad (2.5)$$

Chapter 3 YOLO, SSD and Faster R-CNN

3.1 Overview

Three widely discussed object detection approaches are:

- YOLO
- SSD
- Faster R-CNN

These approaches differ in their detection architectures and computational characteristics.

3.2 YOLO

YOLO is a one-stage object detector.

It performs object localization and classification within a unified detection pipeline.

The architecture is particularly relevant to applications where efficient object detection and low latency are important considerations.

3.3 SSD

SSD stands for Single Shot MultiBox Detector.

It is also a one-stage object detection architecture. SSD performs object detection at multiple feature scales and was designed to provide a balance between detection capability and computational efficiency.

3.4 Faster R-CNN

Faster R-CNN is a two-stage object detection architecture.

The first stage generates candidate object regions and the second stage performs classification and bounding-box refinement.

The additional processing stages make its computational pipeline different from lightweight one-stage detectors such as YOLO.

3.5 Qualitative Comparison

Table 3.1: Qualitative comparison of object detection approaches

Model	Architecture	General Characteristic	Charac- teristic	Real-Time Consid- eration
YOLO	One-stage	Unified object detec- tion pipeline		Designed with fast detection applica- tions in mind
SSD	One-stage	Multi-scale object de- tection		Can support real- time applications depending on config- uration and hardware
Faster R-CNN	Two-stage	Region proposal fol- lowed by classifica- tion and refinement		Generally involves greater computa- tional processing

3.6 Experimental Scope

The experimental implementation in this project is limited to YOLO11n.

No measured SSD or Faster R-CNN results are claimed.

A fair numerical comparison would require all three approaches to be tested using the same:

- Dataset.
- Hardware.
- Input resolution.
- Evaluation protocol.

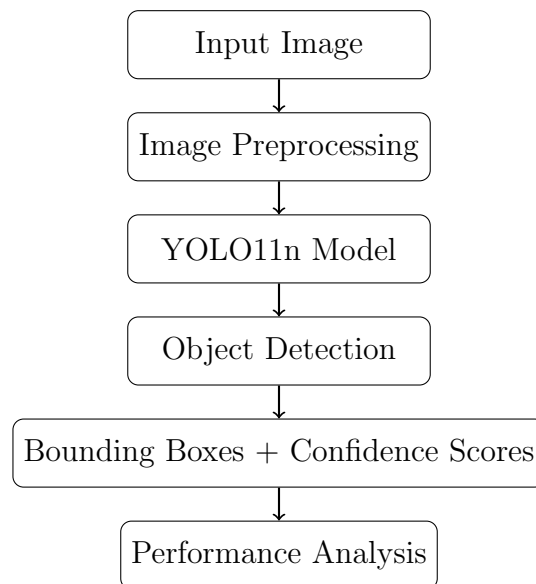
- Confidence threshold.
- Accuracy metrics.
- Performance measurement procedure.

Therefore, the comparison presented in this report is qualitative rather than a fabricated numerical benchmark.

Chapter 4 Methodology

4.1 System Architecture

The implemented object detection pipeline consists of several stages:



4.2 Input Image

The experiment uses an urban vehicle scene containing a bus and pedestrians.

The input image is processed by YOLO11n and the resulting detections are visualized using bounding boxes and confidence scores.

4.3 Detection Process

The detection process consists of the following steps:

1. Load the YOLO11n model.

2. Load the input image.
3. Perform YOLO inference.
4. Extract bounding boxes.
5. Extract predicted object classes.
6. Extract confidence scores.
7. Generate an annotated image.
8. Save the detection output.
9. Store detection results in CSV format.

4.4 Performance Measurement

Repeated inference was performed to obtain performance measurements.

A total of 20 inference runs were used for the benchmark.

The average inference time was calculated using:

$$T_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N T_i \quad (4.1)$$

where $N = 20$.

Approximate FPS was calculated as:

$$FPS_{\text{avg}} = \frac{1}{T_{\text{avg}}} \quad (4.2)$$

Chapter 5 Experimental Setup

5.1 Hardware Environment

The experiment was performed using Google Colab with an NVIDIA Tesla T4 GPU.

Table 5.1: Experimental environment

Parameter	Value
Platform	Google Colab
GPU	NVIDIA Tesla T4
Framework	Ultralytics
YOLO Model	YOLO11n
Ultralytics Version	8.4.164
PyTorch Version	2.11.0+cu128
Programming Language	Python
Image Processing	OpenCV
Data Analysis	Pandas
Visualization	Matplotlib
Benchmark Runs	20

5.2 Software Components

The major software components used in the project are:

- Python for implementation.
- Ultralytics for YOLO inference.
- OpenCV for image processing.
- Pandas for storing and analyzing detection results.
- Matplotlib for visualization.
- Google Colab for execution.

5.3 Project Structure

The final project repository is organized as follows:

```
YOLO-Autonomous-Vehicle/
|
|-- README.md
|-- requirements.txt
|-- .gitignore
|-- model_comparison.md
|-- COLAB_INSTRUCTIONS.md
|-- YOLO_Autonomous_Vehicle_Project.ipynb
|
|-- input/
|   |-- vehicle_scene.jpg
|
|-- output/
|   |-- vehicle_detection.jpg
|
|-- results/
|   |-- vehicle_detection_results.csv
|   |-- experiment_summary.csv
|   |-- experiment_summary.txt
|   |-- graphs/
|
|-- src/
|   |-- detect_image.py
|   |-- performance_analysis.py
|
|-- report/
|   |-- Project_Report.pdf
```

Chapter 6 Results and Analysis

6.1 Object Detection Result

The YOLO11n model successfully processed the urban vehicle scene.

The detected objects included:

- Bus
- Person
- Person
- Person
- Person

The bus received a confidence score of approximately:

$$Confidence_{\text{bus}} \approx 0.940 \tag{6.1}$$

6.2 Detection Visualization

Figure 6.1 shows the annotated output generated by YOLO11n.

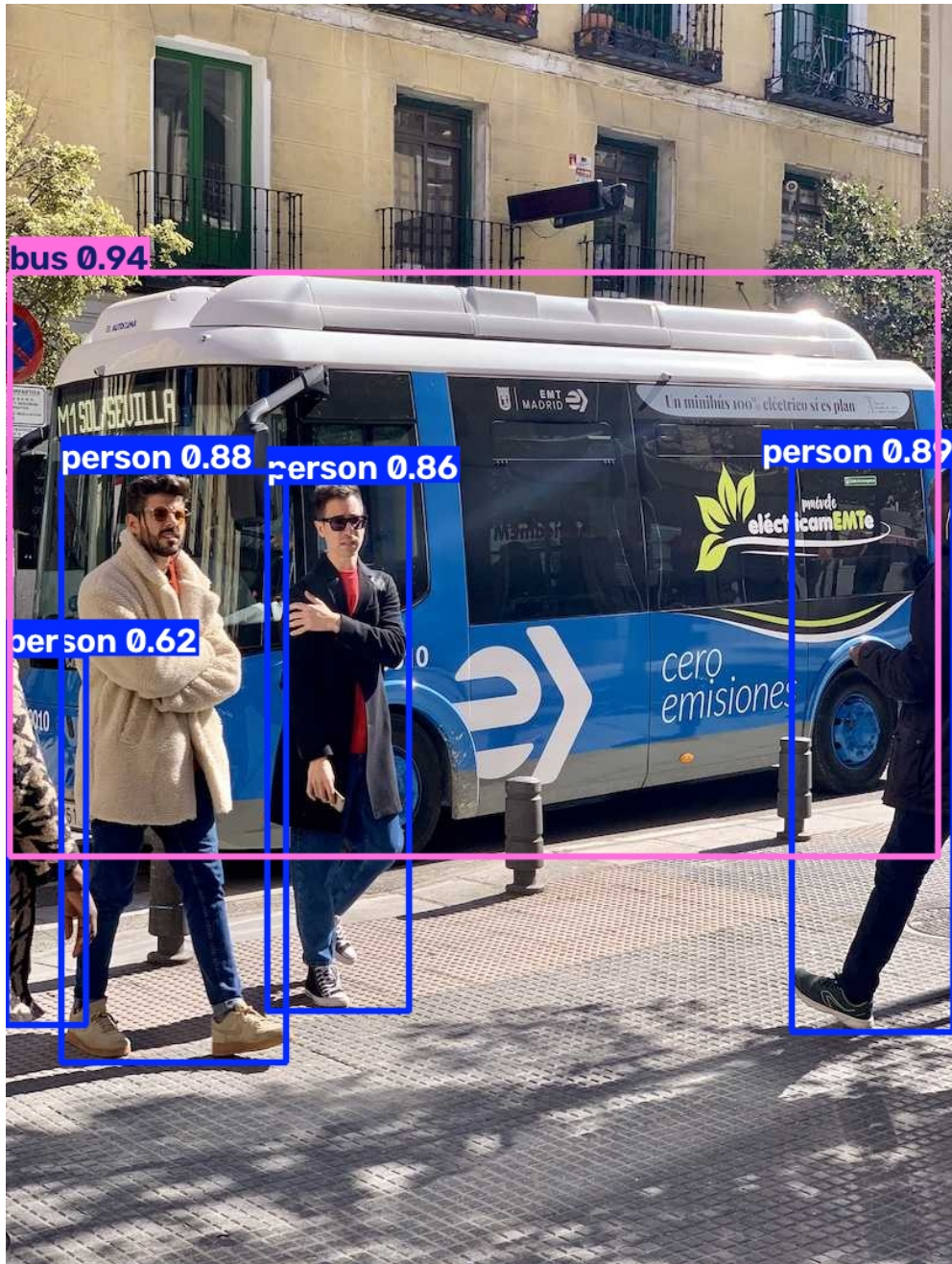


Figure 6.1: YOLO11n detection result for the urban vehicle scene

6.3 Detection Confidence Results

The recorded confidence values were approximately:

Table 6.1: Detected objects and confidence scores

Object	Confidence
Bus	0.940
Person	0.888
Person	0.878
Person	0.856
Person	0.622

The confidence results indicate that the model produced a high-confidence prediction for the bus and relatively high-confidence predictions for the pedestrians in the tested scene.

6.4 Confidence Score Visualization

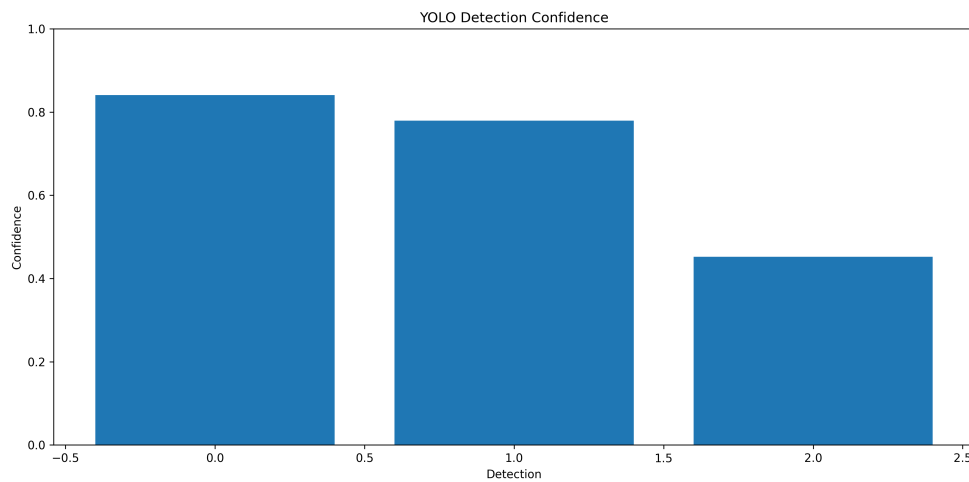


Figure 6.2: Confidence scores of detected objects

6.5 Single Inference Performance

During one vehicle-scene inference run, the measured inference time was:

$$T = 0.0626 \text{ seconds} \quad (6.2)$$

The corresponding approximate FPS was:

$$FPS = \frac{1}{0.0626} \approx 15.97 \quad (6.3)$$

This is an individual inference measurement and is different from the 20-run benchmark described below.

6.6 20-Run Performance Benchmark

A 20-run benchmark was performed to obtain a more representative measurement of inference performance.

Table 6.2: 20-run YOLO11n performance benchmark

Metric	Measured Value
Number of runs	20
Average inference time	0.0202 s
Minimum inference time	0.0184 s
Maximum inference time	0.0249 s
Standard deviation	0.0016 s
Approximate average FPS	49.53

6.7 Inference Time Visualization

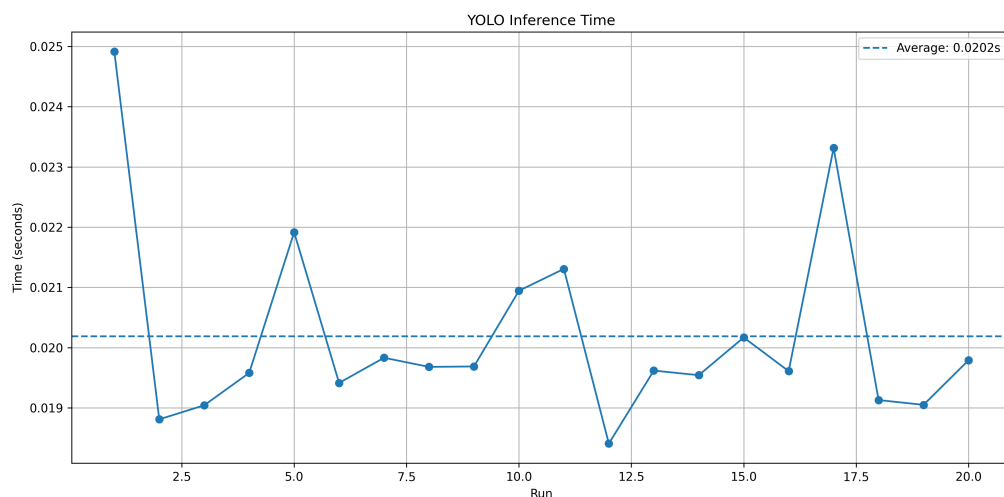


Figure 6.3: Inference time measurements

6.8 FPS Performance

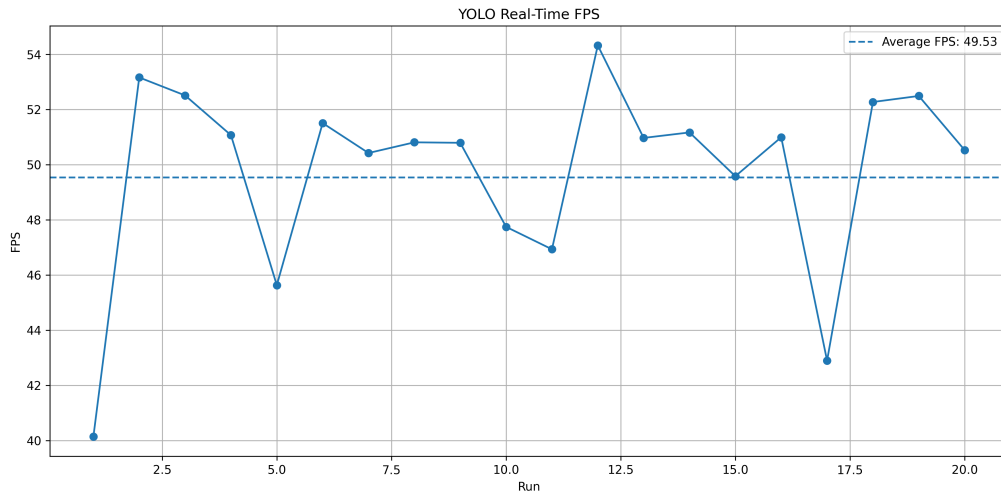


Figure 6.4: FPS performance during repeated inference

The benchmark produced an approximate average throughput of 49.53 FPS in the tested Google Colab Tesla T4 environment.

This demonstrates that the lightweight YOLO11n configuration can perform rapid inference under the tested conditions.

However, the result should not be interpreted as a guaranteed production autonomous-vehicle frame rate because actual system performance depends on hardware, image resolution, model settings, camera processing and other system components.

6.9 Object Distribution

The object distribution generated during the experiment is shown in Figure 6.5.

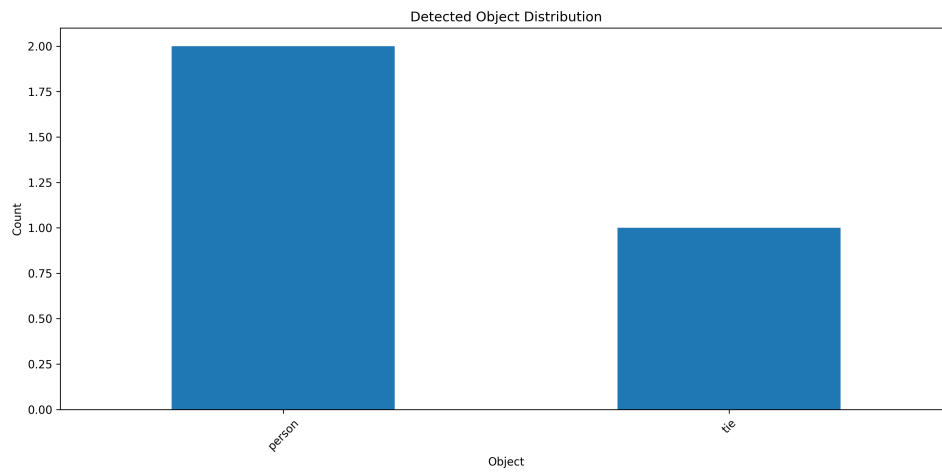


Figure 6.5: Distribution of detected object classes

Chapter 7 Discussion

7.1 Detection Performance

The experiment successfully detected the bus present in the urban scene with a confidence score of approximately 0.940.

The model also identified multiple pedestrians with confidence scores ranging from approximately 0.622 to 0.888.

This demonstrates the basic ability of YOLO11n to identify and localize multiple objects within the same image.

7.2 Real-Time Considerations

The repeated benchmark produced an average inference time of approximately 0.0202 seconds.

The corresponding approximate throughput was:

$$FPS \approx 49.53 \tag{7.1}$$

The measured throughput demonstrates the speed characteristics of the tested lightweight model and hardware combination.

However, inference speed alone does not determine whether a complete autonomous vehicle perception system is suitable for deployment.

A real autonomous vehicle perception system would also need to consider:

- Camera acquisition latency.
- Image preprocessing.
- Data transfer.

- Post-processing.
- Object tracking.
- Sensor fusion.
- Decision-making.
- Hardware reliability.
- Environmental conditions.

7.3 YOLO and SSD

YOLO and SSD are both one-stage object detection approaches.

Both approaches can be considered for applications where detection speed is an important requirement.

The actual performance depends on the specific model configuration, dataset, image resolution and hardware.

Therefore, this project does not claim a universal numerical performance advantage of one architecture over the other.

7.4 YOLO and Faster R-CNN

Faster R-CNN follows a two-stage detection approach.

Its region-proposal stage and subsequent classification and refinement stage create a different computational pipeline from YOLO.

For applications with strict latency requirements, computational complexity is an important consideration when selecting a detection architecture.

This project does not provide a measured Faster R-CNN benchmark and therefore does not claim a numerical performance difference.

7.5 Suitability for Autonomous Vehicle Perception

The experiment demonstrates that YOLO11n can perform object detection with low measured latency in the tested environment.

This makes YOLO-based detection relevant to real-time perception research.

However, autonomous vehicles are safety-critical systems. A production perception system would require extensive testing across different:

- Weather conditions.
- Lighting conditions.
- Road environments.
- Traffic densities.
- Camera viewpoints.
- Object sizes.
- Hardware platforms.

The current experiment therefore represents a project-level demonstration rather than a production-ready autonomous vehicle perception system.

Chapter 8 Limitations

8.1 Limited Dataset

The experiment uses an urban vehicle scene rather than a large, labeled autonomous-driving dataset.

Therefore, the results cannot be used to estimate general detection accuracy across all road environments.

8.2 No Ground-Truth Accuracy Evaluation

The current experiment does not calculate standard dataset-level metrics such as:

- Precision.
- Recall.
- Mean Average Precision (mAP).
- F1-score.

These metrics require a suitable labeled evaluation dataset.

8.3 Hardware Dependency

The measured FPS was obtained using an NVIDIA Tesla T4 GPU in Google Colab.

Performance can vary on other GPUs, CPUs and embedded computing platforms.

8.4 Image-Based Evaluation

The current implementation focuses on image inference rather than a complete continuous camera-video pipeline.

A real autonomous vehicle would continuously process a sequence of frames.

8.5 Limited Model Comparison

Although YOLO, SSD and Faster R-CNN are discussed in the project, only YOLO11n was experimentally executed.

A complete comparative study would require implementing and evaluating all three approaches under identical conditions.

Chapter 9 Future Scope

The project can be extended in several directions.

9.1 Larger Autonomous Driving Datasets

Future experiments can use larger labeled autonomous-driving datasets.

Such datasets would allow quantitative evaluation using precision, recall and mAP.

9.2 Video-Based Detection

The image-based system can be extended to real-time video.

A video perception pipeline can be represented as:

$$Frame_t \rightarrow YOLO \rightarrow Detection_t \rightarrow Tracking \rightarrow Frame_{t+1} \quad (9.1)$$

This would provide a more realistic representation of continuous vehicle perception.

9.3 Object Tracking

Object tracking can be integrated to maintain object identities across consecutive frames.

Tracking could provide information about:

- Object movement.
- Object trajectories.
- Relative motion.
- Temporal consistency.

9.4 Full Model Comparison

A future experiment can implement:

- YOLO.
- SSD.
- Faster R-CNN.

on the same dataset and hardware.

The following metrics could then be compared:

- mAP.
- Precision.
- Recall.
- Inference latency.
- FPS.
- Memory usage.

9.5 Edge Hardware Deployment

Future work can investigate deployment on embedded hardware used in robotics and autonomous systems.

This would provide a more realistic evaluation of computational requirements outside a cloud GPU environment.

9.6 Sensor Fusion

Camera-based object detection can eventually be combined with LiDAR and radar information.

Sensor fusion could provide additional information about object distance, depth and motion.

Chapter 10 Conclusion

This project implemented YOLO11n for object detection in an urban vehicle scene as part of an autonomous vehicle perception study.

The system successfully detected a bus and multiple pedestrians and generated bounding boxes and confidence scores for the detected objects.

The highest-confidence bus detection was approximately 0.940.

Performance testing was also conducted.

A single measured inference produced an inference time of 0.0626 seconds, corresponding to approximately 15.97 FPS.

A separate 20-run benchmark produced an average inference time of approximately 0.0202 seconds and an approximate average throughput of 49.53 FPS in the tested NVIDIA Tesla T4 Google Colab environment.

The experiment demonstrates the potential of lightweight YOLO detection for low-latency object detection tasks.

However, the results are limited to the experimental environment and scene used in this project. A production autonomous vehicle perception system would require more extensive evaluation using large labeled datasets, multiple environmental conditions, quantitative accuracy metrics, continuous video streams and dedicated automotive hardware.

The project therefore provides a practical foundation for further investigation of deep-learning-based perception systems for autonomous vehicles.

Bibliography

- [1] Ultralytics, *Ultralytics YOLO Documentation and Framework*.
- [2] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- [3] W. Liu et al., “SSD: Single Shot MultiBox Detector,” European Conference on Computer Vision (ECCV), 2016.
- [4] S. Ren, K. He, R. Girshick and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” Advances in Neural Information Processing Systems, 2015.
- [5] OpenCV, *Open Source Computer Vision Library*.
- [6] Google, *Google Colaboratory*, Cloud-based computational notebook environment.

Chapter A Experimental Results Summary

Table A.1: Complete experimental summary

Parameter	Value
Model	YOLO11n
Platform	Google Colab
GPU	NVIDIA Tesla T4
Ultralytics Version	8.4.164
PyTorch Version	2.11.0+cu128
Test Scene	Urban bus / vehicle scene
Bus Confidence	0.940
Person Confidence 1	0.888
Person Confidence 2	0.878
Person Confidence 3	0.856
Person Confidence 4	0.622
Single Inference Time	0.0626 s
Single Inference FPS	15.97
Benchmark Runs	20
Average Inference Time	0.0202 s
Minimum Inference Time	0.0184 s
Maximum Inference Time	0.0249 s
Standard Deviation	0.0016 s
Average Benchmark FPS	49.53

Chapter B Project Repository

The complete implementation is maintained in the **YOLO-Autonomous-Vehicle** project repository.

The repository contains:

- Source code.
- Google Colab notebook.
- Input image.
- Annotated detection output.
- Detection result CSV files.
- Performance measurements.
- Generated graphs.
- Project documentation.